

Course Information

CSE 5095-001: Special Topics in Comp. Sci. & Engineering: Types and Programming Languages

Instructor: Stefan Muller, stefan.muller@uconn.edu

Lectures: Tu/Th 9:30–10:45 AM, SHH 107

Prerequisites: CSE 3502 (Theory of Computation) or equivalent is strongly recommended. In particular, I will assume some familiarity with mathematical logic, discrete math appropriate for theoretical computer science, and proof by induction. Students who have taken CSE 4102 and/or 5102 will have some background that will be helpful, but this will not be assumed. Undergraduates and non-CSE students with sufficient background are welcome but need instructor permission to register.

Textbooks: (Not required, but useful for reference)

- Robert Harper, *Practical Foundations for Programming Languages (Second Edition)*, Cambridge, 2016.
- Benjamin C. Pierce, *Types and Programming Languages*, MIT, 2002.

Course Description

Many useful features of modern programming languages, such as anonymous functions (lambdas), generics, and ownership, have arisen out of research into the fundamentals of programming languages. This course will introduce students to those fundamentals: students will learn the tools and techniques used by programming languages researchers to model programs, type systems, and the design choices made in the creation of programming languages. There will be a particular focus on type systems, which provide many programming languages with a certain guarantee of runtime safety: usually that type errors will not occur at runtime, but more advanced type systems can guarantee information security, data-race-freedom and many other desirable properties. Students will see how type systems are designed, and how these safety properties of typed programs are proven. The course will also explore some applications of type systems in modern programming language research and practice.

Course Objectives

After taking this class, students should be able to:

- Understand the models, notations and techniques used by Programming Languages researchers to formalize languages and systems.
- Formally prove properties about programming languages and programs.
- Use the above techniques to design new type systems and models to formalize and prove properties about new and existing language features and systems.
- Read and understand professional literature in the field of Programming Languages.

Grading

Grading will be based on:

- 35% Assignments (roughly 6)
- 20% Midterm Exam
- 20% Reading and Presenting a Research Paper
- 25% Final Exam

Final letter grades will be assigned based on the Graduate Grading Scale at <https://registrar.uconn.edu/grades/> (e.g., A represents “work of distinction”, B represents “work of good quality”, etc.) I will assign these after all grades are in based on the performance of the class and the quality of student work; I don’t have a set cutoff for what constitutes an A grade, nor do I have a set maximum number of each grade I’m willing to assign.

Policies

Late Days/Late and Missing Work

Each student has 6 “late days” to use on homeworks over the course of the semester. Using a late day extends the deadline by 24 hours. You do not need to let me know you’re using late days; they’ll be deducted automatically when you hand in the work.

Important: No more than two late days may be used on any one assignment. Late days may not be used on exams or presentations.

After 48 hours, late work will not be accepted without the instructor’s permission, which must be obtained before the deadline (and will be granted only for good reasons, e.g., medical emergencies).

If you’ve used up your late days, work up to 2 days late (without prior permission from the instructor) will be accepted with a penalty of 10% per day.

If, due to illness or emergency, you will need to miss the midterm exam or in-class presentation, let the instructor know as soon as possible to discuss whether it is possible to make up the assessment. Final Exam attendance policies are handled by the Registrar.

Collaboration, AI and Outside Sources

In this course, we will be learning new techniques for solving problems and proving things about your solutions. If you use these techniques in your research after taking the course (and I hope you do to some extent!) it will probably be on difficult, novel problems where it will require some ingenuity to apply them; researchers in that situation may enlist the help of others (and, now, of generative AI) either to assist in the new applications or to perform the more mundane parts of the development so they can focus on the big ideas. In order to ask the right questions and evaluate the appropriateness of the answers though, you need to have a basic understanding of the techniques in the first place. That’s the understanding this class provides, and it can only come from thinking about how to apply the techniques yourselves. I’ve developed this policy with that in mind.

Collaboration between students to understand the material and general strategies for solving homework questions is permitted, as is the use of generative AI tools and outside sources (e.g., other textbooks, lecture notes from other courses) for the purposes of enhancing your understanding of lecture material. However, **all work** submitted for credit (homework, presentations, and exams) must be your own individual work, unless specified otherwise (e.g., for a collaborative presentation), and you are responsible for its correctness and originality. Some examples of violations of this policy are sharing answers with other students (this is a violation for all students involved, even if the answers were the original work of one of the students), looking up answers to questions on the web, and submitting a solution any part of which was written by another student, someone else, or an AI tool. Note that these are just some examples, but not an exhaustive list. If you have any doubt as to whether something is allowed by the policy, feel free to ask the instructor. For any

assignments specified as collaborative, all team members are of course expected to collaborate in producing the submission; the rest of the policy (including with respect to collaborations with others outside the team and AI) still applies. For more information, see UConn's Academic Integrity Policy.

If you have any doubts about the policy, it's best to contact me. I may issue changes and clarifications as the semester goes on.

Despite the limited use of AI tools allowed by the policy above, the course is designed to be accomplished without the use of AI—no AI use is required and my intention is that students who choose not to use it should not be at a disadvantage. In the spirit of abiding by this policy myself, I generally do not use AI in developing course materials such as notes, slides, homeworks or exams. I may occasionally use AI to develop extra convenience materials, such as practice problems, but will always make an effort to check the result myself, and will disclose the use of AI to you in these cases.

Students with Disabilities

The University of Connecticut is committed to protecting the rights of individuals with disabilities and assuring that the learning environment is accessible. If you anticipate or experience physical or academic barriers based on disability or pregnancy, please let me know immediately so that we can discuss options. Students who require accommodations should contact the Center for Students with Disabilities, Wilbur Cross Building Room 204, (860) 486-2020 or <http://csd.uconn.edu/>.

Policy Against Discrimination, Harassment and Related Interpersonal Violence

The University is committed to maintaining a safe and non-discriminatory learning, living, and working environments for all members of the University community – students, employees, or visitors. Academic and professional excellence can flourish only when each member of our community is assured an atmosphere of mutual respect. All members of the University community are responsible for the maintenance of an academic and work environment in which people are free to learn and work without fear of discrimination or discriminatory harassment. In addition, inappropriate amorous relationships can undermine the University's mission when those in positions of authority abuse or appear to abuse their authority. To that end, and in accordance with federal and state law, the University prohibits discrimination and discriminatory harassment, as well as inappropriate amorous relationships, and such behavior will be met with appropriate disciplinary action, up to and including dismissal from the University. Additionally, to protect the campus community, all responsible employees (including faculty), as outlined in the Policy Against Discrimination, Harassment and Related Interpersonal Violence, are required to report to the Office of Institutional Equity any information that they receive related to sexual assaults, intimate partner violence, and/or stalking involving a student. An exception to this reporting exists if students disclose information as a part of coursework submitted to an instructor in connection with a course assignment. Even in the absence of such obligation, all Employees are encouraged to contact OIE if they become aware of information that suggests a safety risk to the University community or any member thereof. The University takes all reports with the utmost seriousness. Please be aware that while the information you provide will remain private, it will not be confidential and will be shared with university officials who can help. More information, including confidential and exempt employee resources available for support and assistance, can be found at <http://equity.uconn.edu> and <http://titleix.uconn.edu>.

Topics

(Tentative and subject to change; we may not cover all of these)

- Proofs by Rule Induction
- Fundamentals of PL Models
 - Abstract Syntax
 - Dynamic Semantics
 - Static Semantics
 - Type Safety
 - Binding and Substitution
- Modeling Imperative Languages
- The Untyped Lambda Calculus
- The Simply-typed Lambda Calculus
 - Sum and Product Types
 - The Curry-Howard Correspondence
- Polymorphism, Parametricity and Abstraction
- General Recursion and Recursive Types
- Subtyping
- Modeling Mutable State
- Models of Language Evaluation
 - Big-step Semantics
 - Cost Semantics
 - Evaluation Contexts
 - Control Stacks
- Other Evaluation Styles
 - Lazy Evaluation
 - Parallelism and Concurrency
 - Continuations and Continuation Passing Style
- Applications
- Possible Advanced Topics (based on time and interest)
 - Higher Kinds
 - Type and Effect Systems
 - Substructural Type Systems
 - Dependent Types